Introduction to Software Engineering

CSE 362

Software Development and Professional Practice

Winter 2024

© Royal Military College of Canada

A bit of history…

- 1940s: First computers were built, since then the field has developed tremendously.

- Applications from the early years were:
  - Quite small.
  - Written by one person
  - Written and used by the experts in the application area concerned.
  - Problems to be solved were mostly of a technical nature.
  - Emphasis was on expressing known algorithms efficiently in a programming language.
  - Input/output mostly numeric

A bit of history…

- In the 1960s:
  - Software has grown both in size and complexity
  - But to many, programming was still an *art*, and had never become a *craft*.
  - Most programmers learned by doing – no formal education in the field.

- Problems started arising:
  - Late delivery.
  - Programs not behaving as expected.
  - Programs not easily adaptable
  - Errors and bugs.

→ Software crisis

A bit of history …

- Two conferences were devoted to the topic in 1968 and 1969
  - Naur and Randell, 1968
  - Buxton and Randell 1969

- As a result, the term software engineering was coined in a somewhat provocative senses
  - Shouldn't it be possible to build software in the way one builds bridges and houses?
  - i.e. having in place a theoretical basis + sound design and construction techniques.

What is Software Engineering?

An early definition was given at the first NATO conference (Naur and Randell, 1968):

> *Software engineering is the establishment and use of sound engineering principles in order to obtain economically software that is reliable and works efficiently on real machines.*

Software engineering vs. programming?

- Software engineering certainly is *not* the same as programming
- Programming is *an important ingredient* of software engineering.
  - Yes, we are concerned about the correctness of programs.

- But, we also care about:
  - Building *large* programs
  - Software that *evolves*
  - Software built with *efficiency*
  - Regular *cooperation* between people
  - *Supporting users* effectively
  - Building software *for other fields*/cultures

Viewing software engineering as a branch of engineering

Yes, there is an overlap

- In both cases we work from a set of desired functions, using scientific and engineering techniques in a creative way
- development of the product in a number of phases, a careful planning of these phases, continuous audit of the whole process, construction from a clear and complete design, etc.

Viewing software engineering as a branch of engineering

But could also be problematic:

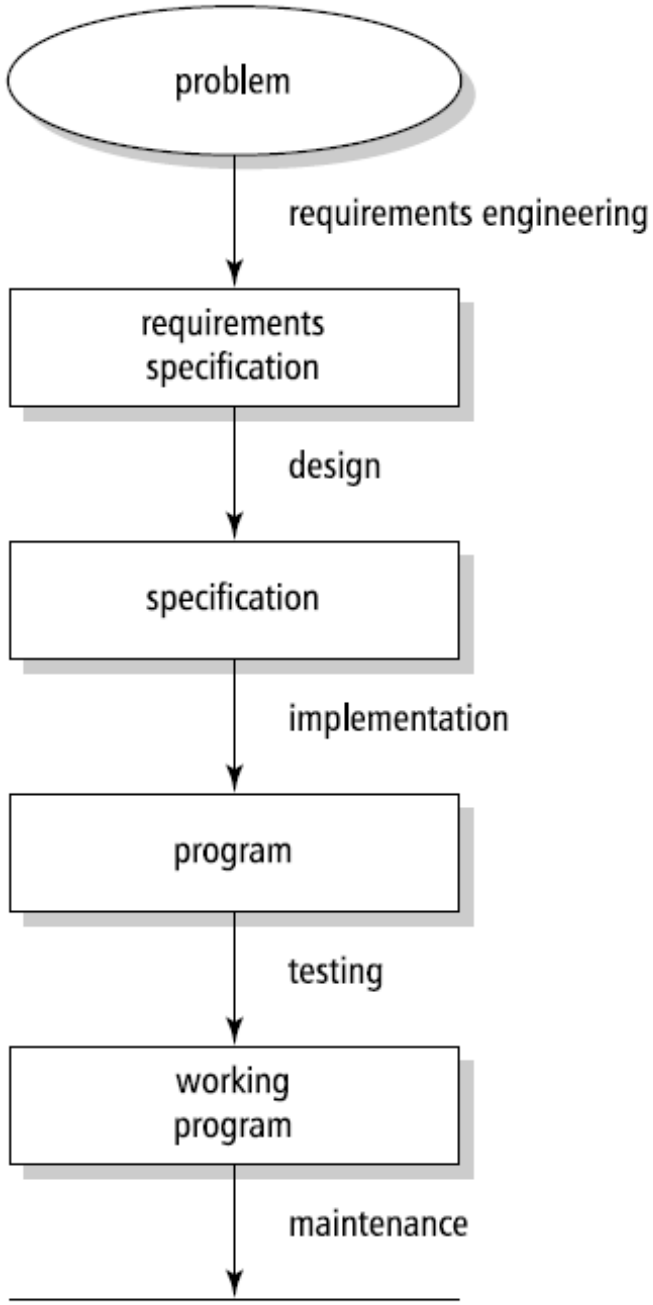
- The engineering metaphor hints that we deal with clearly defined needs/problems, that can be fulfilled if we follow all the right steps
- The assumption that we are dealing with well-defined problems in SE simply does not hold.
- Design is open-ended. The solution emerges as we go along.

Phases in the Development of Software

Building a house analogy

- When building a house, the builder does not start with piling up bricks.
- The requirements and possibilities of the client are analyzed first
  - Taking into account hobbies, finances, family size, etc.
- The architect takes these factors into consideration to come up with a design.
- Design approved. Construction starts

A simple view of software development



Requirements engineering

- A complete description of the problem to be solved

- the requirements posed *by the environment* in which the system is going to function, e.g. *hardware, supportive software, # of users, etc.*

- the requirements posed *on the environment* in which the system is going to function, e.g. *does new hardware need to be purchased to satisfy the requirements?*

- Output document: the **requirements specification**

Design

- During the design phase (crucial), a model of the whole system is developed which, when encoded in some programming language, solves the problem for the user.

- The problem is decomposed into manageable pieces called **components.**

- The functions of these components and the **interfaces** between them are specified in a very precise way.

- Separate the *what* from the *how*

Implementation

- concentrate on the individual components.
- development of a well-documented, reliable, easy to read, flexible, correct, program.
- The result of the implementation phase is an executable program.

Testing

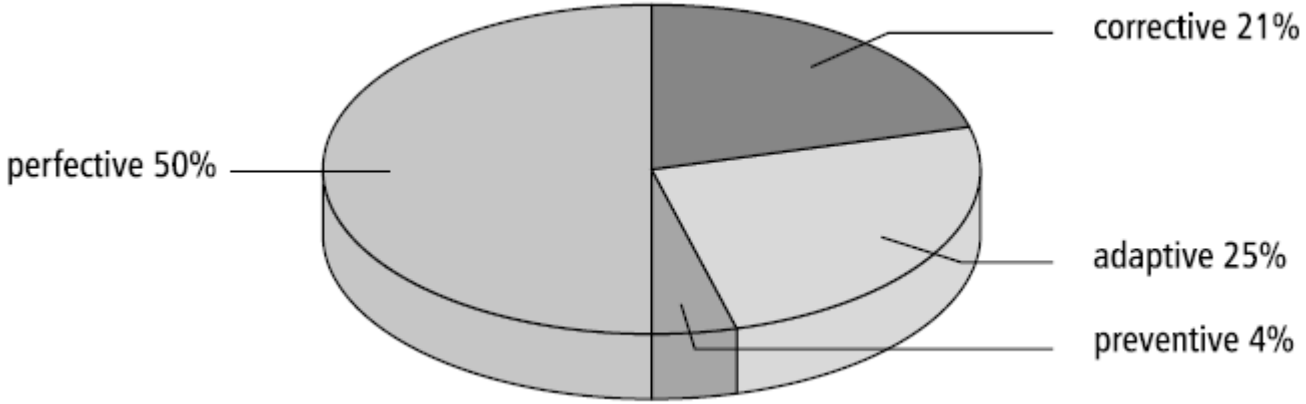
- In reality testing occurs at every stage of software development: The earlier that errors are detected, the cheaper it is to correct them.

- There are two flavours to testing:
  - Verification: checking for correctness
  - Validation: checking that we are fulfilling user requirements

Maintenance

- After delivery of the software, there are often errors that have still gone undetected.
- Also, the actual use of the system can lead to requests for changes and enhancements.

→ Maintenance takes place, i.e. keeping the system operational after it has been delivered to the user.



Other activities

- An activity spanning all phases is **project management,** *e.g. project planning, team organization, quality issues, cost and schedule estimation*

- An important activity not identified separately is **documentation**, *e.g., project plan, quality plan, requirements specification, architecture description, design documentation and test plan*

Historical Software Failures

- Ariane 5, Flight 501
- Therac-25
- The London Ambulance Service

- **And more ….**

What is going on now in SE?

- The rise of agile methods
- There is shift from *producing* software to *using* software
- Software development is becoming more heterogeneous: no one controls all.